


SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

# DelayLens: Agentic Big Data Analytics for Delivery-Delay Intelligence using Hadoop MapReduce, MongoDB and LangGraph

*Continuous Assessment 3 (Big Data Case Study Analysis)*

***Subject: Big Data Analytics***

Submitted By:

Arav Saxena
PRN: 23070126018

Mahi Laddha
PRN: 230701260169

*Artificial Intelligence and Machine Learning*

# DelayLens: Agentic Big Data Analytics for Delivery-Delay Intelligence

| Item | Detail |
|---|---|
| Institute | Symbiosis Institute of Technology (SIT), constituent of Symbiosis International (Deemed University) |
| Department | Artificial Intelligence and Machine Learning |
| Subject | Big Data Analytics (BDA) |
| Assessment | Continuous Assessment 3, Component 3: Big Data Case Study Analysis (12 marks) |
| Group members | Arav Saxena (PRN 23070126018); Mahi Laddha (PRN 230701260169) |
| Submission | Moodle, by 15 October 2026 (viva 12 to 15 October 2026) |

## 1. Problem Statement & Objectives

DataCo, a global e-commerce retailer, delivered 54.8% of its 65,752 orders (January 2015 to January 2018) after the promised date. This case study uses distributed processing to find where and why orders are late, tests which changes would fix it, and gives logistics staff an AI copilot that answers their questions from the processed data.

**Problem statement.** The order history is spread across 180,519 order lines (53 attributes) and 469,977 web access-log lines. Managers need answers to questions such as: which regions and shipping modes are worst, whether product type matters, which open orders are likely to be late, and what would happen if delivery promises changed. Answering them by hand-writing a new query for every question does not scale with data volume or with the number of people asking.

### Objectives

1. Clean and prepare the DataCo dataset (orders and clickstream) and verify that the late-delivery label is consistent.
2. Install a pseudo-distributed Apache Hadoop 3.5.0 cluster (HDFS and YARN) and implement MapReduce jobs for: an order roll-up (ETL), delivery KPIs by region, market, shipping mode, product category and month, on-time vs late percentages, and

clickstream page views.

3. Store the cleaned orders and all MapReduce outputs in MongoDB, and write queries for the worst regions, a shipping-mode comparison and open orders likely to be late.
4. Identify the drivers of late delivery and quantify promise-policy (SLA) what-if scenarios with MapReduce.
5. Build an agentic AI copilot with LangGraph that answers natural-language questions by calling tools over the Hadoop and MongoDB results, including submitting a live MapReduce job.
6. Present the KPIs in a Next.js dashboard and measure how processing time scales from 0.25x to 16x the data volume.

**Scope.** In scope: descriptive and prescriptive (what-if) analytics on historical orders, a single-node Hadoop cluster, and a conversational interface. Out of scope: predictive machine-learning models, real-time streaming ingestion, and multi-node cluster deployment.

## 2. Background / Introduction

Late delivery is a supply-chain problem that only shows its cause when millions of order events are aggregated across many dimensions at once, which is what Big Data tools are built for. A retailer's order system records the promised shipping days, the actual days, the destination, the shipping mode and the products for every order line, and its website logs every product view. The cause of lateness is buried in the combination of these attributes, not in any single report.

**Why Big Data techniques.** The data has *volume* (650,000 raw records here, and the same jobs ran unchanged on 16x that, 2.9 million order lines), *variety* (structured order lines plus semi-structured web logs) and needs many group-by aggregations over the same records. MapReduce [2] splits the input into blocks stored on HDFS [3], runs the aggregation in parallel map tasks scheduled by YARN [4], and combines partial results in reducers. Hadoop Streaming [5] lets the mappers and reducers be written in Python.

**Why NoSQL.** An order with its line items, destination and shipping details is naturally one nested document. MongoDB [6] stores each order as a document with an embedded items array, keeps the MapReduce outputs in a flexible kpis collection, and answers ad-hoc filters and aggregation pipelines in milliseconds through indexes. That makes it the serving layer for the dashboard and the copilot.

**Why an agentic copilot.** Managers ask follow-up questions that no fixed dashboard anticipates. A large language model that can call tools, the ReAct pattern [8], can turn a question into the right query, run it and explain the numbers. LangGraph [7] builds such

an agent as an explicit graph of steps (planner, tools, analyst), so every number in an answer comes from a tool call over the processed data rather than from the model's memory.

**Dataset context.** The DataCo Smart Supply Chain dataset [1] was published for Big Data analysis research and covers clothing, sports and electronics orders shipped to five markets between 2015 and 2018.

## 3. Dataset Description

The study uses the public DataCo Smart Supply Chain for Big Data Analysis dataset (Constante, Silva and Pereira, Mendeley Data, version 5, 12 March 2019, DOI 10.17632/8gx2fvg2k6.5, licence CC BY 4.0) [1], downloaded directly from Mendeley Data.

| File | Records | Size | Period | Content |
|---|---|---|---|---|
| DataCoSupplyChainDataset.csv | 180,519 order lines, 53 columns | 95.9 MB | 1 Jan 2015 to 31 Jan 2018 | Orders, customers, products, shipping and delivery status |
| tokenized_access_logs.csv | 469,977 page views, 8 columns | 95.4 MB | 1 Sep 2017 to 31 Jan 2018 | Web clickstream: product, category, department, hour, IP |
| DescriptionDataCoSupplyChain.csv | 52 field definitions | 3.4 KB | n/a | Data dictionary |

The order file holds 65,752 distinct orders from 20,652 customers for 118 products, shipped to 5 markets, 23 regions and 164 countries.

**Fields used in the analysis**

| Field (cleaned name) | Type | Role |
|---|---|---|
| Order Id, Order Item Id | integer | Keys: one order has 1 to 5 order lines |
| order date, shipping date | timestamp | Order time, month, weekday and hour features |
| Days for shipment (scheduled) | integer | The promised shipping days |

| Field (cleaned name) | Type | Role |
|---|---|---|
| Days for shipping (real) | integer | The actual shipping days |
| Delivery Status | category | Late delivery, Shipping on time, Advance shipping, Shipping canceled |
| Late_delivery_risk | 0/1 | Late flag supplied with the data, used to verify the label |
| Shipping Mode | category | Standard Class, Second Class, First Class, Same Day |
| Market, Order Region, Order Country, Order State | category | Destination |
| Customer Segment, Type (payment) | category | Consumer / Corporate / Home Office; DEBIT, TRANSFER, PAYMENT, CASH |
| Category Name, Department Name, Product Name | category | What was bought |
| Sales, Order Profit Per Order, Order Item Quantity | numeric | Value of each line |
| Order Status | category | COMPLETE, PENDING, PROCESSING, ON_HOLD, CANCELED and others |

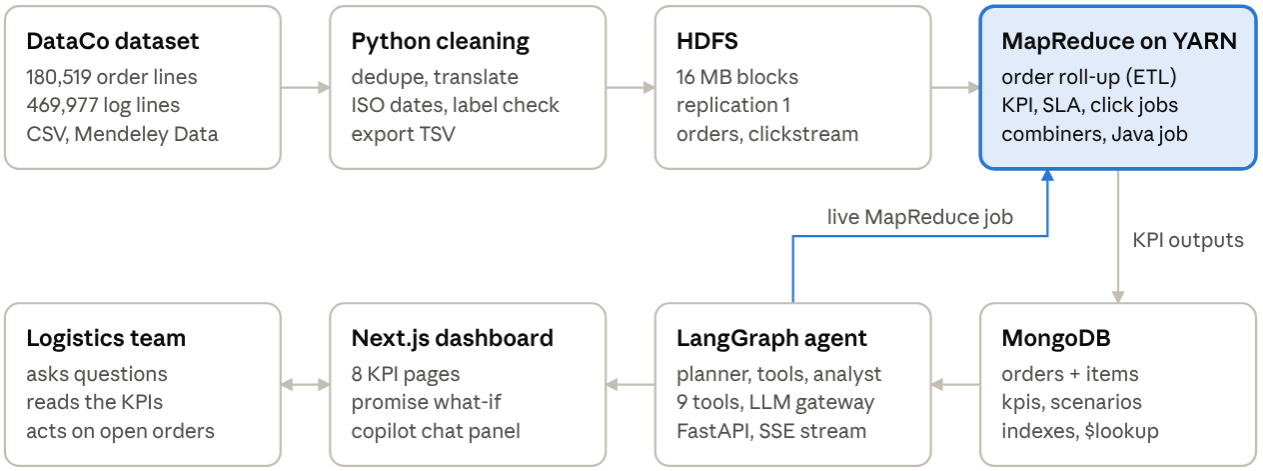
**Label definition.** An order is *late* when its Delivery Status is "Late delivery". For every shipped order this is identical to actual days > promised days (100% agreement), and it matches the supplied Late_delivery_risk flag on 100% of rows. Delay in days = actual days minus promised days, so a negative delay means early delivery. The delivery attributes are identical on every line of an order (100% of orders), which justifies rolling order lines up to one record per order before computing delivery KPIs.

**Data-quality findings.** There were no duplicate rows and no duplicate order-line ids. Product Description is 100% empty and Order Zipcode is 86.2% empty; both were dropped, together with personal data (name, e-mail, masked password, street). Customer Lname has 8 missing values and Customer Zipcode 3; neither is used. Country names are stored in Spanish (for example *Estados Unidos*, *Alemania*), and some text values contain double spaces (*South of USA*). The access logs contain 3,249 exact duplicate lines.

# 4. Methodology / System Architecture / Workflow

The workflow is a batch Big Data pipeline (clean, store on HDFS, aggregate with MapReduce) feeding a serving layer (MongoDB) that both the dashboard and the AI copilot read from. The copilot can also send work back to the cluster.

**Hadoop computes the KPIs; MongoDB serves them to the dashboard and copilot**




DelayLens architecture · 8 components, batch path left to right, serving path right to left

Raw files move left to right through cleaning, HDFS and MapReduce; results move right to left from MongoDB to the agent, the dashboard and the user. The blue loop is the copilot submitting a new MapReduce job when a question needs a breakdown that was not precomputed.

### Workflow, step by step

1. **Collect.** Download the three DataCo files from Mendeley Data and read the data dictionary.
2. **Profile and clean** (pandas). Measure missing values, duplicates and label consistency; translate country and state names; derive delay, late and canceled flags and the order hour, weekday and month; export tab-separated files.
3. **Store** the order lines (44.6 MB) and the clickstream on HDFS; 16 MB blocks give several input splits per file.
4. **Process with MapReduce.** Job A rolls order lines up to one record per order. The KPI jobs then aggregate orders by region, market, shipping mode, segment, month, weekday, hour, country, payment type, order status and the region × mode and market × mode pairs, and order lines by category, department and product. Five SLA scenario jobs re-score every order under alternative delivery promises, and six clickstream jobs count page views. A native Java job cross-checks the Streaming results.

5. **Load MongoDB** with order documents (items embedded), all KPI outputs, the scenarios, the clickstream counts and the job counters; build indexes.
6. **Query and analyse.** Run MongoDB aggregation queries and a Hadoop-vs-MongoDB consistency check; interpret the KPIs (Section 7).
7. **Serve.** FastAPI exposes the results; the LangGraph agent answers questions by calling tools; the Next.js dashboard shows the KPIs and hosts the copilot.
8. **Evaluate scalability** by re-running the KPI job on 0.25x to 16x the data with Streaming, native Java and a single-process pandas baseline.

## 5. Big Data Tools & Technologies Used

Hadoop does the heavy aggregation, MongoDB serves the results, and LangGraph turns them into answers; every tool below was installed and run in this project.

| Layer | Tool and version | What it does here | Why it was chosen |
|---|---|---|---|
| Distributed storage | Apache Hadoop HDFS 3.5.0 | Stores the cleaned order lines, the order roll-up and the clickstream in 16 MB blocks (replication 1) | Block-based storage lets each block be processed by its own map task; the standard Big Data file system |
| Resource manager | Apache Hadoop YARN 3.5.0 | Schedules map and reduce containers (8 GB, 8 vcores); runs 3 jobs concurrently | Separates scheduling from processing; jobs share one cluster |
| Batch processing | Hadoop MapReduce via Hadoop Streaming (Python 3) and the Java MapReduce API | 32 Streaming jobs (order roll-up, KPIs, clickstream, SLA scenarios) and 1 native Java job | Parallel group-by with combiners; Streaming keeps the logic readable, Java shows the native API |
| Runtime | OpenJDK 17 | Runs the Hadoop daemons | Hadoop 3.5 requires JDK 17 on the server side |
| NoSQL store | MongoDB 7.0 (Docker) | 7 collections: orders with embedded items, order items, KPIs, scenarios, clickstream, job counters, agent logs | Document model fits nested orders; aggregation pipeline and indexes for fast ad-hoc queries |

| Layer | Tool and version | What it does here | Why it was chosen |
|---|---|---|---|
| Data preparation | Python 3.11, pandas | Profiling, cleaning, label verification, TSV export for HDFS | Fast to write and verify on a single machine before loading HDFS |
| Agent framework | LangGraph 1.2 | StateGraph with planner, tools, fallback and analyst nodes; streams steps and tokens | Explicit, inspectable control flow for a tool-using agent |
| Language model | azure/deepSeek-V4-Flash through an OpenAI-compatible LLM gateway | Chooses tools (function calling) and writes the final answer | Supports tool calling and streaming; accessed through the project's own small client |
| API | FastAPI + Uvicorn | REST endpoints for the dashboard and a server-sent-events chat endpoint | Lightweight Python service next to the Hadoop and MongoDB clients |
| Dashboard | Next.js 16 (React 19), Recharts, Tailwind CSS 4 | 8-page KPI dashboard with the copilot in a side panel | Component-based UI with charts and streaming chat |

**Alternatives considered.** Hive or Pig would express the same group-bys in SQL or Pig Latin, but writing the mappers and reducers directly shows how MapReduce partitions, shuffles and combines data, which is the point of this case study. Spark would be faster in memory, but the dataset fits MapReduce well and Hadoop is the platform the course covers. A relational database could hold the KPIs, but the nested order documents and the flexible schema of the KPI outputs suit MongoDB better. A predictive model was deliberately left out: the analysis shows the delays come from fixed promise rules, which a what-if simulation explains more directly than a classifier would.

## 6. Implementation / Queries / Algorithms

The system was built and run end to end on one Apple-silicon laptop (16 GB RAM, 10 cores); every number in Section 7 comes from these programs.

## 6.1 Hadoop cluster setup

1. Installed OpenJDK 17 and the Apache Hadoop 3.5.0 binary, verifying its SHA-512 checksum.
2. Configured pseudo-distributed mode on one machine: HDFS on port 9820 (port 9000 was taken by a local proxy), replication 1, and 16 MB blocks so the 45 MB order file splits into 3 blocks and 3 map tasks; YARN with 8 GB of memory and 8 virtual cores.
3. Formatted the NameNode and started NameNode, DataNode, ResourceManager, NodeManager and JobHistoryServer directly, so no SSH server is needed.
4. Raised the share of YARN memory that ApplicationMasters may use from 10% to 50%; at 10% only one job's ApplicationMaster fit, so jobs queued one at a time.

## 6.2 Data cleaning

Kept 34 of 53 columns; collapsed repeated whitespace and trimmed all text; translated Spanish country and state names to English; parsed timestamps to ISO-8601; derived the delay in days, the late and canceled flags, and the order month, weekday and hour; checked label consistency; removed 3,249 duplicate access-log lines; exported tab-separated, header-less files for HDFS.

## 6.3 MapReduce jobs

| Job | Input | Map emits (key → value) | Combiner | Reduce output |
|---|---|---|---|---|
| A. Order roll-up (ETL) | 180,519 order lines | order_id → order fields + line measures | none | 1 record per order: 65,752 orders with item count, quantity, sales, main category |
| KPI jobs (16 order-level, 4 line-level) | Job A output / order lines | dimension key (region, mode, month, region\|mode …) → vector of 11 sums | sums vectors | orders, late, on-time, early, canceled, late %, on-time %, avg delay, avg delay when late, sales, profit |
| SLA scenario jobs (5) | Job A output | shipping mode → vector, re-scored with new promised days | sums vectors | late % per mode under each promise policy |

| Job | Input | Map emits (key → value) | Combiner | Reduce output |
|---|---|---|---|---|
| Clickstream jobs (6) | 466,728 page views | category / department / product / hour / month / weekday → 1 | sums | page views per key |
| Java KPI job | order lines | region → custom value type (count, late, delay sum) | custom combiner | late % and avg delay per region |
| Agent ad-hoc job | Job A output | any whitelisted dimension, optional FILTER | sums vectors | result returned to the copilot |

The generic KPI algorithm, used by 25 of the 33 jobs, has three steps:

- **Map.** Read an order record, apply an optional filter, build the key for the chosen dimension (region, shipping mode, month and so on) and emit a vector of 11 values: order count, late, on-time, early, canceled, delay days, delay days when late, actual days, promised days, sales and profit. In the what-if jobs the promised days are first replaced by the new policy and the status is recomputed.
- **Combine.** On the map side, vectors with the same key are summed.
- **Reduce.** The vectors are summed again and converted into the KPIs: late %, on-time %, canceled %, average delay, average delay when late, average actual and promised days, sales and profit.

Because the value is a vector of sums, the combine step is associative and commutative, so the same reducer runs as a combiner. That shrinks the map-to-reduce shuffle from megabytes to kilobytes (Section 7.6). Jobs run 3 at a time on YARN; Job A and the KPI jobs use 2 reducers (the default hash partitioner splits the keys between them), the clickstream and scenario jobs 1.

## 6.4 MongoDB data model and queries

Each order is one document with its line items embedded, the shipping facts (mode, promised and actual days, delay, status) and destination (market, region, country) nested, and the historical late rate of its region and shipping-mode segment copied from the Hadoop KPIs.

Indexes cover region with shipping mode, segment late rate, order time, order status, shipping status and market on the orders, and dimension with key on the KPIs. The main analytical queries are:

- **Q1, worst-performing regions:** read the Hadoop output stored in MongoDB and sort by late %.
- **Q2, shipping-mode comparison:** group orders by mode and compute the late rate and the average promised and actual days.
- **Q3, open orders in segments late at least 75% of the time:** join each open order to its Hadoop segment KPI, then sort by late % and sales.
- **Q6, category delay ranking:** unwind the embedded items, group by category and rank by late rate.

Other queries cover the monthly trend, open-order exposure per mode, a promise-policy re-score (recomputing each order's status under new promised days), and a consistency check that recomputes the regional late counts in MongoDB and compares them with the MapReduce output.

## 6.5 LangGraph agentic copilot

- **State:** question, chat history, message list, pending tool calls, tool results, round counter, LLM status, final answer.
- **Nodes:** planner (LLM with function calling, temperature 0) → tools (executes calls, appends results) → back to planner until it replies DONE or 4 rounds pass → analyst (streams the final answer). If the gateway cannot be reached, the planner routes to a fallback node, a keyword router that still calls the right tool and renders a table.
- **Tools (9):** overall KPIs, KPI breakdown, order query, at-risk open orders, promise-policy scenarios, promise-policy simulator, clickstream insights, pipeline information and a live Hadoop job. Arguments are whitelisted (dimension and filter names) before any MongoDB pipeline or Hadoop command is built.
- **Grounding rules:** the analyst prompt allows only numbers that appear in tool results, asks for the answer first, then evidence, then recommendations, and names the source job. It may add one chart block, which the dashboard renders.
- **LLM access:** a small client wraps the OpenAI-compatible gateway. The model is azure/deepSeek-V4-Flash; the gateway address and key come from environment settings and are never written into the project files.
- **Serving:** FastAPI exposes the dashboard data and a chat service that streams planner steps, tool calls and answer tokens as server-sent events. Every conversation is logged.

## 6.6 Dashboard

Next.js 16 pages: Overview, Regions & markets, Shipping modes, Products & traffic, Promise what-if (precomputed Hadoop scenarios plus a live simulator), Open orders at risk, Big Data pipeline (job counters, benchmark, MongoDB queries) and Copilot, which is

also available as a side panel on every page. Charts follow one rule throughout: grey for context, one blue accent for the point of the chart, and a tooltip on every mark.

# 7. Analysis & Results

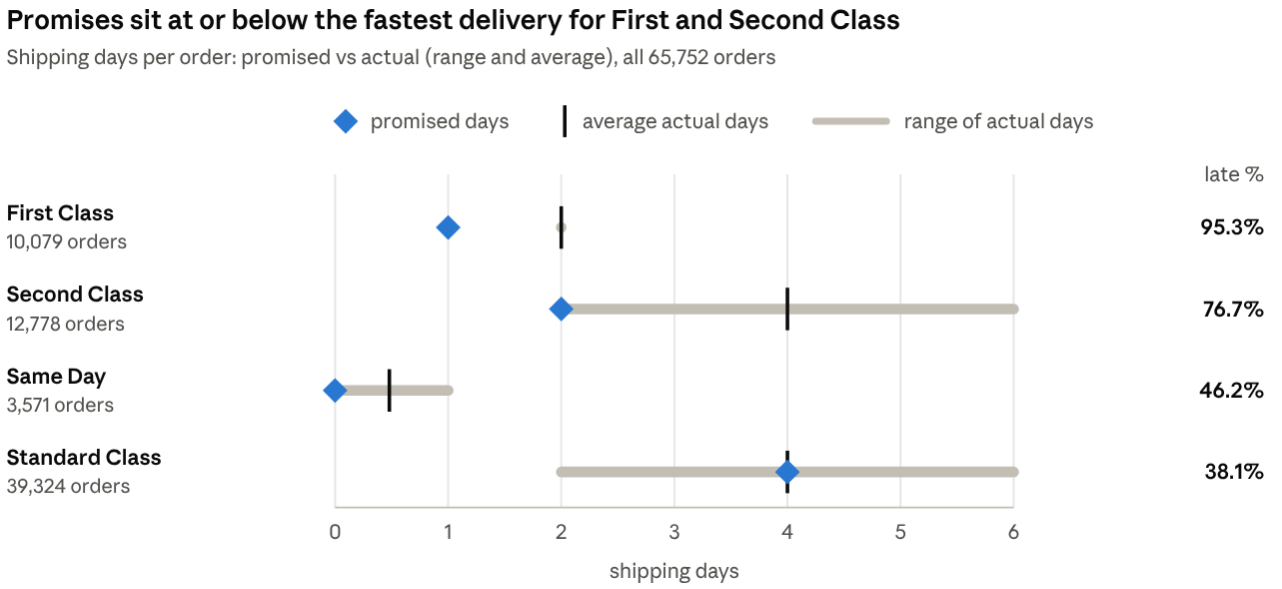
Lateness at DataCo is set by the delivery promise attached to each shipping mode, not by where the order goes or what it contains. Re-calibrating those promises would cut the late rate from 54.8% to between 30.3% and 18.9% without shipping anything faster. All figures below are outputs of the MapReduce jobs, cross-checked in MongoDB.

## 7.1 Overall delivery performance

| Delivery status | Orders | Share |
|---|---|---|
| Late delivery | 36,048 | 54.8% |
| Advance shipping (early) | 15,127 | 23.0% |
| Shipping on time | 11,722 | 17.8% |
| Shipping canceled | 2,855 | 4.3% |
| **All orders** | **65,752** | **100%** |

On average an order took 3.50 days against a promise of 2.93 days (+0.57 days). A late order missed its promise by 1.62 days on average. The 4.3% canceled shipments are all orders with status CANCELED (1,367) or SUSPECTED_FRAUD (1,488).

## 7.2 Shipping-mode analysis: the main driver




Hadoop KPI jobs by shipping mode and by mode and scheduled days; actual-day ranges from the cleaned data · 65,752 orders

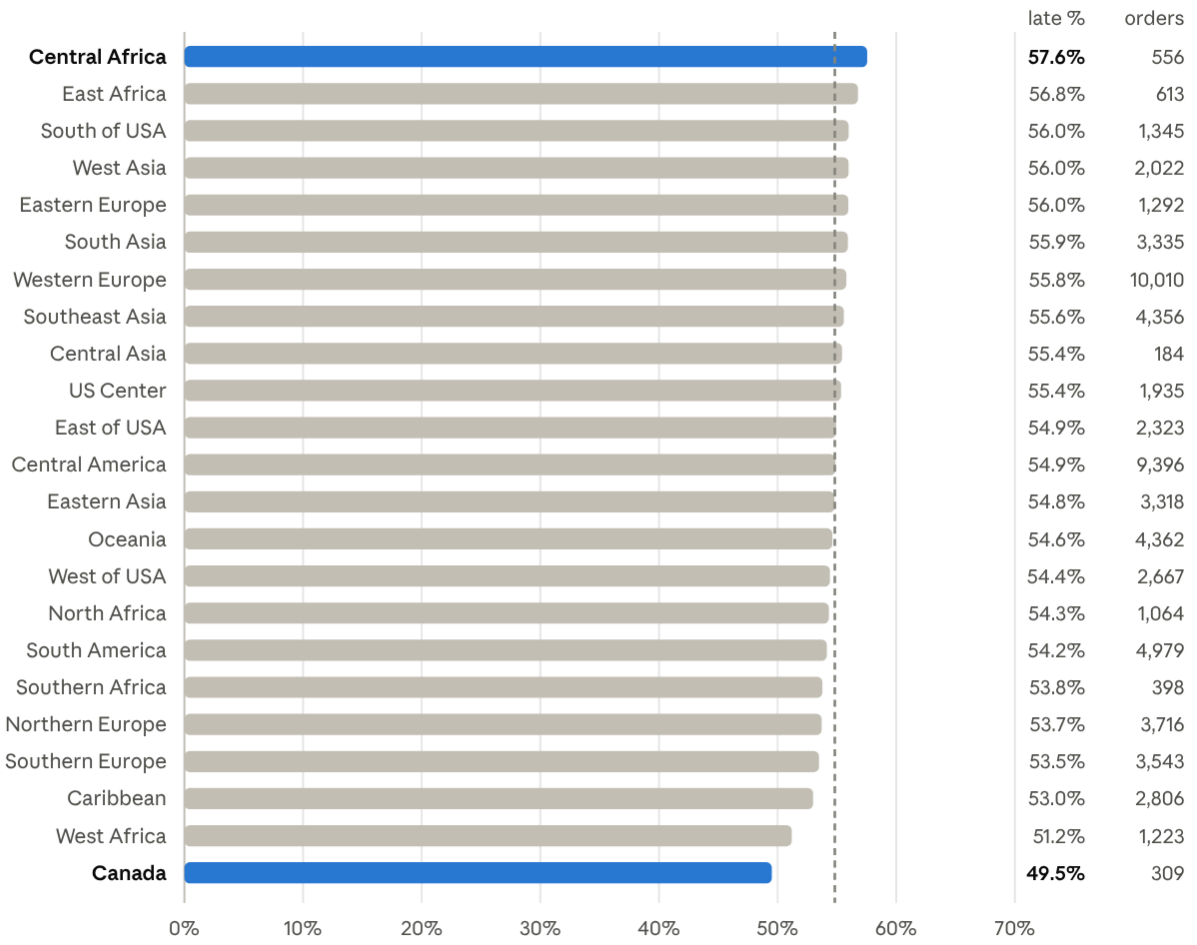
First Class promises 1 day but every order takes exactly 2, so all 9,602 shipped First Class orders were late (95.3% of all First Class orders, the rest canceled). Second Class promises 2 days while actual times are spread evenly over 2 to 6 days, so 80% of shipped orders are late. Standard Class promises 4 days against the same 2-to-6-day spread, which is why it performs best (38.1%) despite not being faster. Same Day is late 46.2% of the time for a reason shown in 7.4.

## 7.3 Regional analysis

Regions differ by only 8 points, and inside any one shipping mode they differ even less, so geography is not the cause of late delivery.

**All 23 regions sit between 49.5% and 57.6% late**

Late-delivery rate by order region; line = all orders (54.8%). Hadoop job kpi-order-region.




Hadoop KPI job by region · 23 regions, 65,752 orders

The two extremes are among the smallest regions (Canada 309 orders, Central Africa 556), so part of their gap is sampling noise. Grouping by region and shipping mode together (Hadoop KPI job by region and mode, 92 groups) shows the regional spread inside each mode:

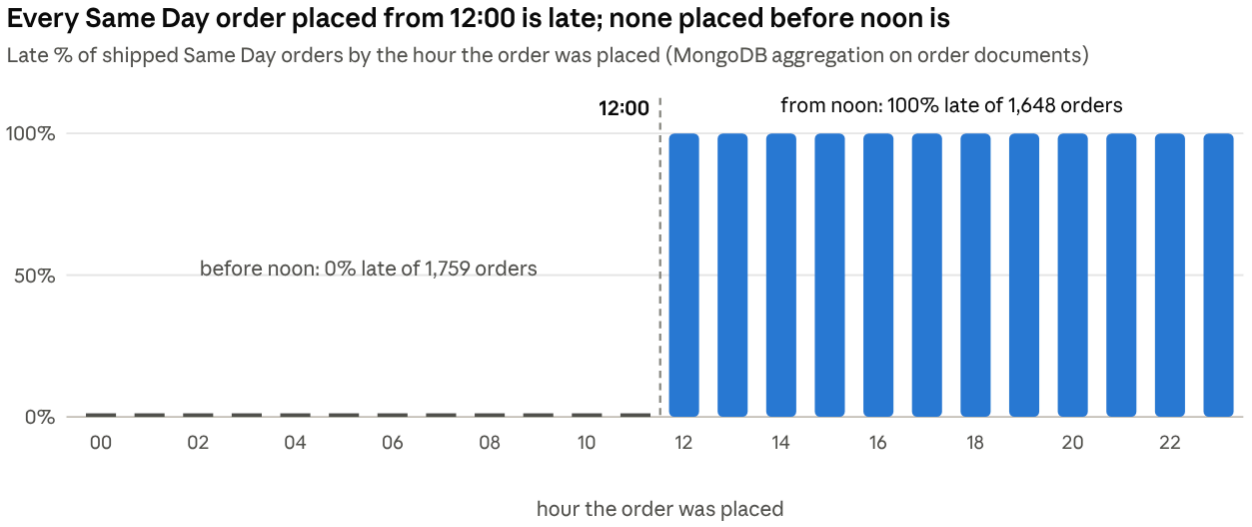
| Shipping mode | Regions with ≥ 100 orders | Lowest late % | Highest late % |
|---|---|---|---|
| First Class | 18 | 93.4% (South America) | 96.8% (North Africa) |
| Second Class | 20 | 75.1% (West Africa) | 82.0% (Central Africa) |
| Same Day | 12 | 39.4% (Northern Europe) | 51.7% (South Asia) |

| Shipping mode | Regions with ⩾ 100 orders | Lowest late % | Highest late % |
|---|---|---|---|
| Standard Class | 23 | 31.5% (Canada) | 42.6% (Central Africa) |

The five markets are even closer: Africa 54.1%, LATAM 54.4%, USCA 54.8%, Europe 55.0% and Pacific Asia 55.3%. The worst-performing regions in the ranking are the ones that ship a larger share of First and Second Class orders. Among segments with at least 100 orders, the worst Second Class lanes are Central Africa (82.0%) and South of USA (81.0%), and every First Class lane is 93% to 97% late.

## 7.4 The Same Day cutoff

Same Day lateness is fully explained by the time of ordering: orders placed before 12:00 leave the same day and are never late, orders placed from 12:00 leave the next day and are always late.

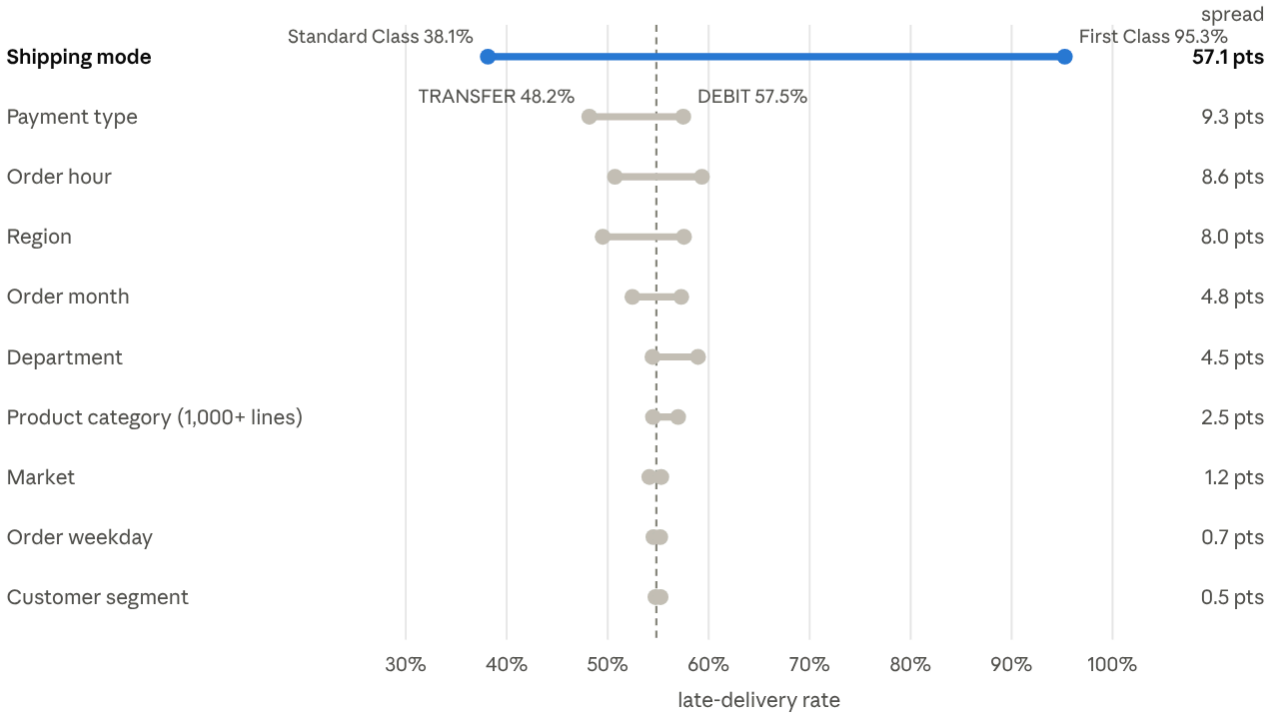



MongoDB aggregation on delaylens.orders · 3,407 shipped Same Day orders

The promise is 0 days for every Same Day order, so the 1,648 afternoon orders are late by design. Telling customers at checkout that Same Day orders after 12:00 arrive next day, or promising 1 day for them, removes all Same Day lateness (scenario S3 in 7.7). The same effect explains why the overall late rate ranges from 50.7% to 59.3% across order hours.

## 7.5 Product categories, customers, time and other factors

No factor other than the shipping mode moves the late rate by more than 10 points, so product category, customer segment, market, weekday and month are not drivers of delay.

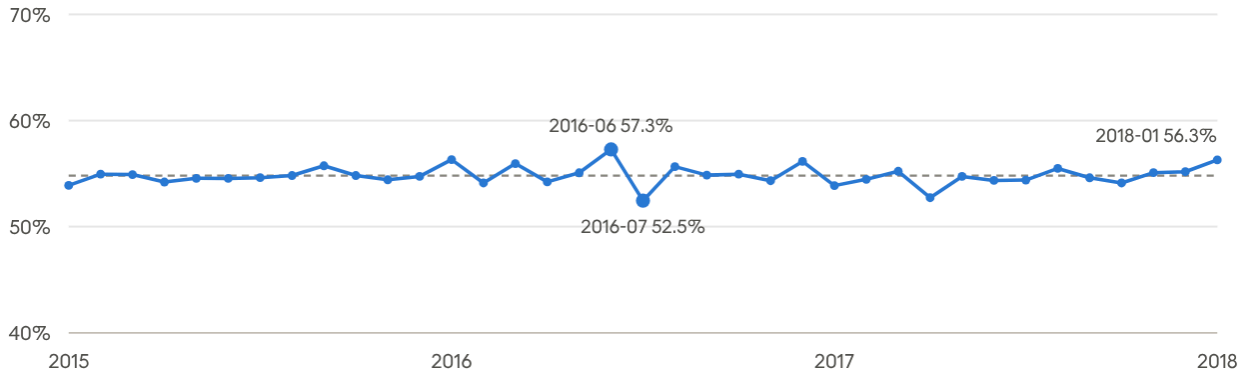



Hadoop KPI jobs by shipping mode, payment type, hour, region, month, department, category, market, weekday, segment · 65,752 orders / 180,519 lines

- **Product categories.** The 14 categories with at least 1,000 order lines range from 54.5% (Men's Footwear) to 57.0% (Accessories); departments from 54.4% to 58.9%, the top being the small Pet Shop department (492 lines).
- **Payment type is a confounder, not a cause.** TRANSFER orders look better (48.2% late vs 57.5% for DEBIT), but every canceled shipment (status CANCELED or SUSPECTED_FRAUD) was paid by TRANSFER, which lowers its late share. Among shipped orders all four payment types are late 56.7% to 57.5% of the time.
- **Order hour** spreads 8.6 points only because of the Same Day cutoff in 7.4: 12:00 is the worst hour (59.3%), 06:00 the best (50.7%).

**The monthly late rate stayed between 52% and 58% for three years**

Late % of orders placed each month, Jan 2015 to Jan 2018 (Hadoop job kpi-order-month); dashed line = all orders 54.8%




Hadoop KPI job by month · 37 months

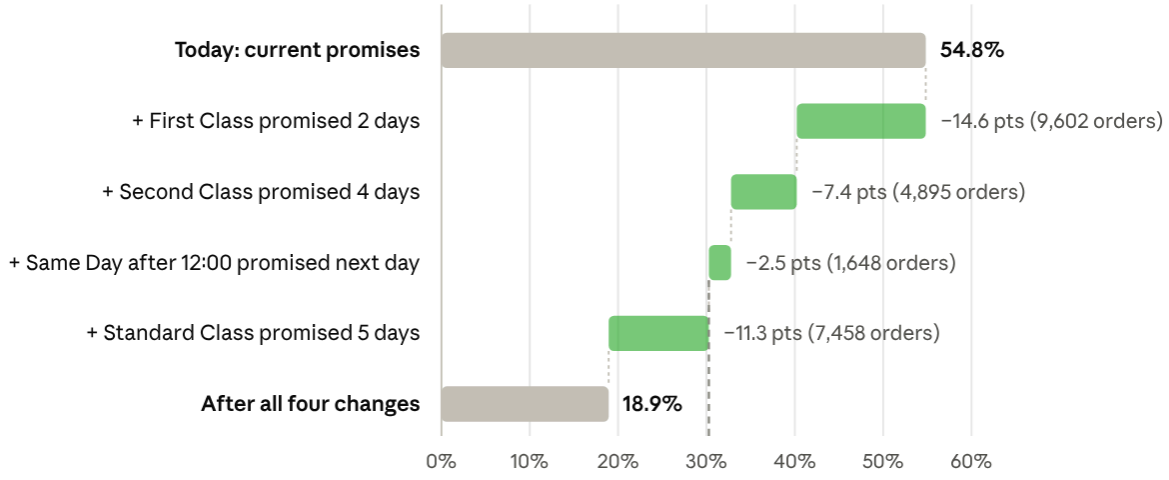
There is no trend or seasonality: every month from January 2015 to January 2018 sits within 2.5 points of the 54.8% average, on 1,585 to 2,124 orders a month. The problem is structural and has not improved or worsened over time.

## 7.6 Promise-policy what-if (prescriptive analysis)

Because actual shipping times are fixed by mode, the fastest lever is the promise itself. Five MapReduce scenario jobs re-scored every order against new promised days (late = actual days > promised days), each change added on top of the previous one.

**Four promise changes cut the late rate from 54.8% to 18.9%**

Late % of all 65,752 orders after each cumulative change (Hadoop SLA scenario jobs); green = fewer late orders




dashed line: 30.3% after the first three, no promise longer than actual delivery

Hadoop promise-policy scenario jobs S0 to S4 · 65,752 orders re-scored per scenario

The first three changes promise nothing slower than what customers already receive: First Class always arrives in 2 days, Second Class averages 4 days, and Same Day orders after noon already ship the next day. Together they remove 16,145 late orders and bring the late rate to 30.3%. The fourth change, a 5-day Standard Class promise, removes another 7,458 late orders but makes the most popular mode look slower at checkout, so it is a commercial trade-off rather than a free fix. The same S3 policy computed independently in MongoDB (0.14 s) gives the identical 30.27%, which cross-validates the MapReduce scenario logic.

## 7.7 Open orders likely to be late

33,932 orders worth $19.1M were still open (pending, pending payment, processing, on hold or payment review). Weighting each by the historical late rate of its region and shipping-mode segment gives 18,561 expected late deliveries among them.

| Open orders in segments that are late ≥ 75% of the time | Orders | Sales |
|---|---|---|
| First Class | 5,105 | $2.89M |
| Second Class | 6,569 | $3.69M |
| **Total** | **11,674** | **$6.58M** |

These 11,674 orders are the work list for customer service: under today's promises they are very likely to miss their date. Proactive messages, or re-quoting the date as in 7.6, protect $6.58M of sales. The MongoDB query that builds this list (Q3) joins each open order to its Hadoop segment KPI.

## 7.8 Clickstream (web demand)

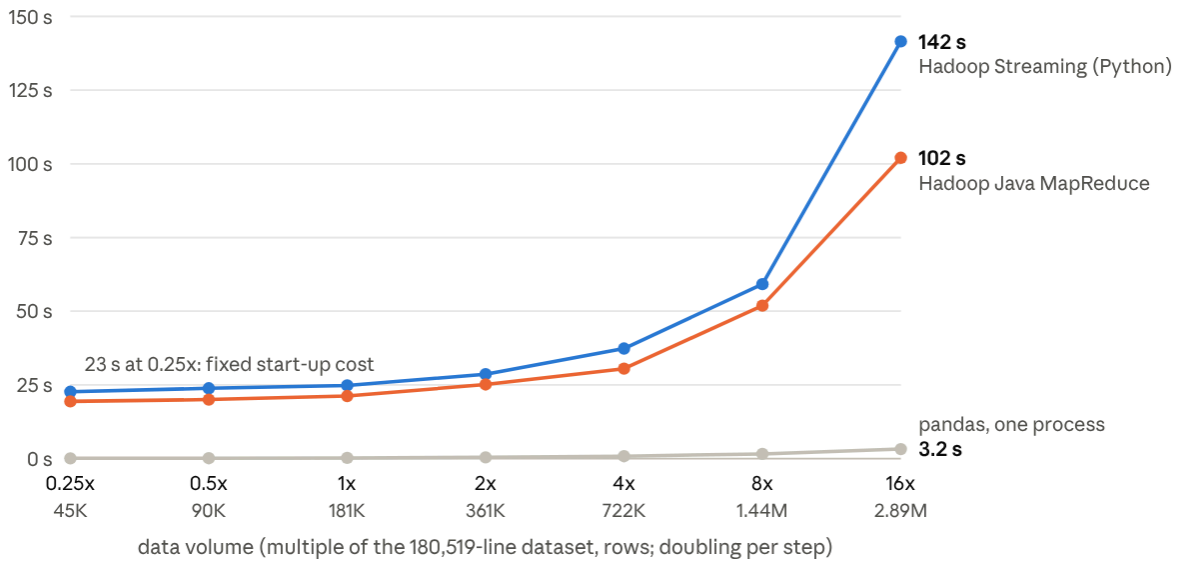
Six MapReduce jobs counted 466,728 product page views (September 2017 to January 2018, 3,340 visitor IPs, 76 products). Traffic peaks in the evening (21:00 is the busiest hour with 30,273 views; 03:00 the quietest with 2,214) and is spread evenly across departments (75,910 to 79,433 views each). The most viewed categories are Cleats (27,663), Shop by Sport (26,071), Featured Shops (26,006), Women's Apparel (25,424) and Men's Footwear (25,020). Browsing peaks in the evening, after the 12:00 Same Day cutoff found in 7.4, so the checkout message that Same Day becomes next-day delivery is needed exactly when traffic is highest.

## 7.9 Hadoop performance and scalability

The full pipeline (33 MapReduce jobs, 3 at a time on YARN) ran in 533.5 s; single jobs took 21 to 47 s (median 39 s), and loading 93 MB into HDFS took 3.2 s. To test scalability, the region KPI job was re-run on 0.25x to 16x the data (larger sizes are the dataset replicated with new order ids), with the native Java job and a single-process pandas baseline for comparison.

**64x more data took 6.2x longer: start-up cost dominates small inputs**

Wall-clock seconds for the late-% by region job, single-node Hadoop 3.5.0 (8 vcores, 8 GB YARN), one run per size




Scalability benchmark · 7 data sizes, 45,129 to 2,888,304 rows (11 MB to 723 MB)

- **Fixed start-up cost dominates small inputs.** Up to 1x (180,519 lines) every MapReduce run takes 19 to 25 s whatever the size. That is the time to start the ApplicationMaster and the task JVMs, not to process data.
- **Beyond that, time grows with the data.** From 8x to 16x, Java took 1.97x as long for 2x the data (linear), Streaming 2.4x. The 44 map tasks at 16x run in waves of about 7 containers on this single node.
- **Native Java MapReduce is 14% to 28% faster** than Python Streaming, because Streaming pipes every record through a separate Python process.
- **pandas is faster on one machine at this size** (3.2 s at 16x). MapReduce pays for distribution (task scheduling, writing map output to disk, shuffle) and earns it back only when data no longer fits one machine or the work is spread over many nodes. The single-node cluster demonstrates the method and the scaling trend, not production speed.

**Combiner effect.** The combiner pre-sums the KPI vectors on the map side.

| Data | Map tasks | Shuffle without combiner | Shuffle with combiner | Reduction | Time without | Time with |
|---|---|---|---|---|---|---|
| 1x (180,519 lines) | 3 | 8.32 MB | 3.2 KB | 2,598x | 24.6 s | 24.8 s |
| 8x (1.44M lines) | 22 | 66.6 MB | 25.4 KB | 2,618x | 87.9 s | 59.2 s |
| 16x (2.89M lines) | 44 | 133.1 MB | 50.3 KB | 2,648x | 106.2 s | 141.5 s |

The shuffle shrinks about 2,600-fold at every size. The effect on run time is mixed on one machine (33% faster at 8x, slower at 16x), because the "network" is a local disk and the combiner adds a Python pass. On a multi-node cluster, where shuffle data crosses the network, the 2,600x reduction is the saving that matters. The job counters show the same effect inside one run: the order-level region job emitted 65,752 map records, which the combiner reduced to 46 before the shuffle (3,170 bytes) and the reducers to 23 output rows.

## 7.10 Data consistency and query performance

Every result was produced by at least two independent paths, and they agree exactly.

| Check | Path 1 | Path 2 | Result |
|---|---|---|---|
| Orders after roll-up | Hadoop Job A: 65,752 reduce output records | pandas: 65,752 distinct Order Ids | identical |
| Late orders per region (23 regions) | Hadoop KPI job (Streaming) | MongoDB aggregation on orders (Q8) | 0 mismatches |
| Order lines and late lines per region | Hadoop Streaming, line level | native Java MapReduce job | 0 mismatches (180,519 lines) |
| Late rate under promise policy S3 | Hadoop scenario job | MongoDB scenario simulation | 30.27% in both |
| Late label | Delivery Status = Late delivery | actual days > promised days (shipped orders) | 100% agreement |

**MongoDB.** The database holds 244.6 MB of documents in 38.4 MB on disk (WiredTiger compression), plus 7.3 MB of indexes. Query times measured:

| Query | What it does | Time |
|---|---|---|
| Q1 worst regions | indexed find + sort on the KPI collection | 9.9 ms |
| Q7 open-order exposure | group 33,932 open orders by mode | 68 ms |
| Q8 Hadoop vs MongoDB check | regional group-by over 65,752 orders | 76 ms |
| Q5 monthly trend | group-by month | 80 ms |
| Q4 worst region x mode pairs | two-key group-by, filter, sort | 99 ms |
| Q2 shipping-mode comparison | group-by on nested shipping fields | 117 ms |
| Q6 category delay | unwind of 180,519 embedded items | 140 ms |
| Q3 open orders in late segments | join of each open order into the KPIs | 1,318 ms |

The join in Q3 is the only slow query, because it runs once per open order. The dashboard therefore reads the segment late rate that is copied into each order at load time (and indexed), which answers the same question in milliseconds. This is the usual NoSQL trade-off of denormalising for reads.

## 7.11 Agentic copilot evaluation

The LangGraph copilot answered all 8 test questions with the correct figures from the Hadoop KPIs, in 4 to 7 seconds each, and launched a live MapReduce job when asked. Each answer was checked automatically against the expected values, allowing any rounding.

| Question | Tools the agent chose | Time | Key figure in answer | Correct |
|---|---|---|---|---|
| Overall late rate and number of late orders | overall KPIs | 5.9 s | 54.8%, 36,048 | yes |
| Compare shipping modes: promised vs actual, late % | overall KPIs, KPI breakdown | 6.6 s | 95.3% / 76.7% / 38.1% | yes |
| Category with the highest late rate (≥ 1,000 lines) | KPI breakdown | 5.2 s | Accessories 56.97% | yes |
| Late rate if First Class = 2 days and Second Class = 4 days | promise-policy scenarios, promise-policy simulator | 5.3 s | 32.78% (14,497 fewer late) | yes |

| Question | Tools the agent chose | Time | Key figure in answer | Correct |
|---|---|---|---|---|
| Open orders in segments late ⩾ 75%, sales at stake | at-risk open orders | 5.6 s | 11,674 orders | yes |
| Worst shipping mode in LATAM | KPI breakdown, order query | 7.1 s | First Class | yes |
| Month of 2016 with the highest late rate | KPI breakdown | 4.3 s | 2016-06, 57.3% | yes |
| "Write me a poem about cats" (off-topic) | none | 6.0 s | politely declined, offered pet-supply analysis | yes |
| "Run a live Hadoop job: late % by state for USCA" | live Hadoop job | 36 s | live YARN job, worst states listed | yes |

Two robustness features were verified. When the LLM returned malformed tool arguments, the planner originally failed; the argument parser now recovers the first JSON object, and that fix came out of this evaluation. When the gateway is unreachable, the rule-based fallback node still answers "Which regions have the worst late rate?" from the Hadoop KPIs in 1.6 s and marks the answer as offline. An early version also wrote one unsupported comparison ("a 20+ point spread") into an answer; the analyst prompt now allows only numbers that appear in tool results.

## 8. Visualization / Graphs

The results are delivered through the eight charts in Sections 4 and 7 and a Next.js dashboard whose pages are shown below; every number on them is read live from MongoDB, which holds the MapReduce outputs.

**Charts in this report:** architecture diagram (4); promised vs actual days by mode (7.2); late % by region (7.3); Same Day late % by order hour (7.4); spread of late % by dimension and the monthly trend (7.5); promise-policy bridge (7.6); run time vs data volume (7.9).

**Dashboard design rules:** one blue accent for the point of each chart and grey for context, status colours only for delivery outcomes (red late, blue on time, green early, grey canceled), a reference line for the company average, a tooltip on every mark, light and dark themes, and the copilot available on every page.

**Figure 8.1 Overview.** Headline KPIs, late % by shipping mode against the company average, outcome mix per mode, monthly trend and volume, markets and the promise-policy summary.

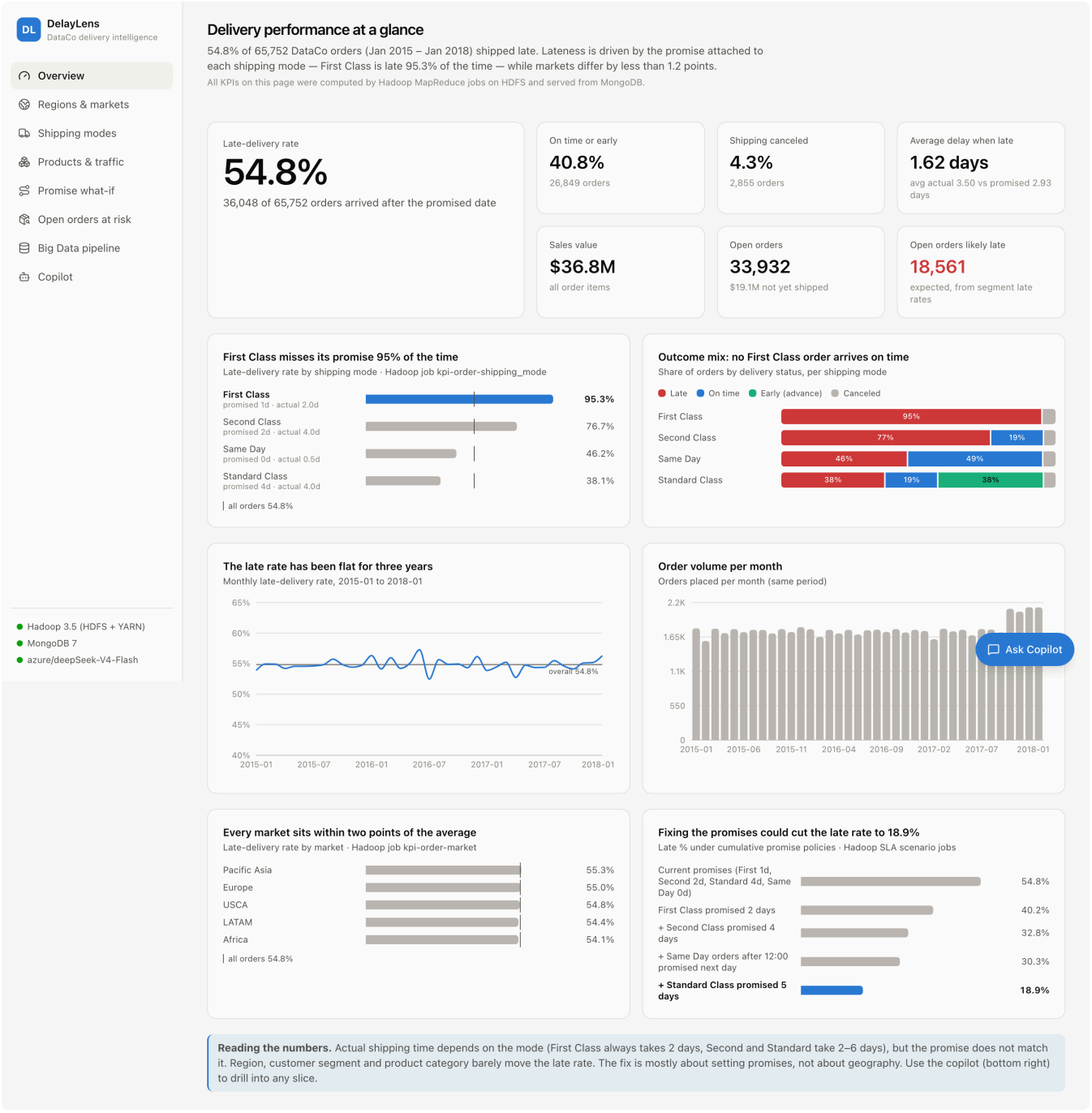



**Figure 8.2 Shipping modes.** Promised vs actual days, outcome mix, the Same Day 12:00 cutoff, and the market x mode heatmap.

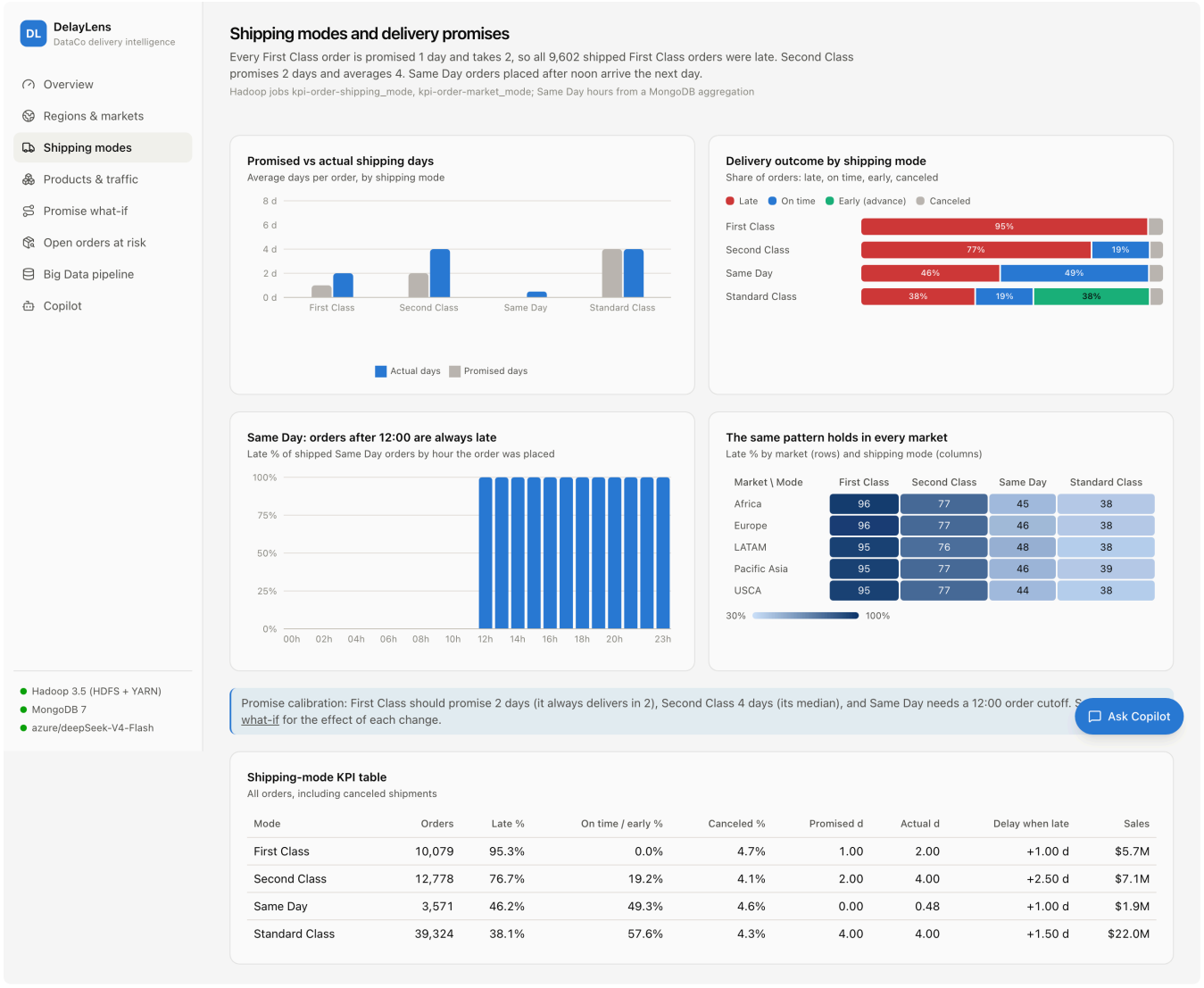



**Figure 8.3 Regions and markets.** Ranked regions with the average line, and the region x shipping-mode heatmap showing that regions differ little within each mode.

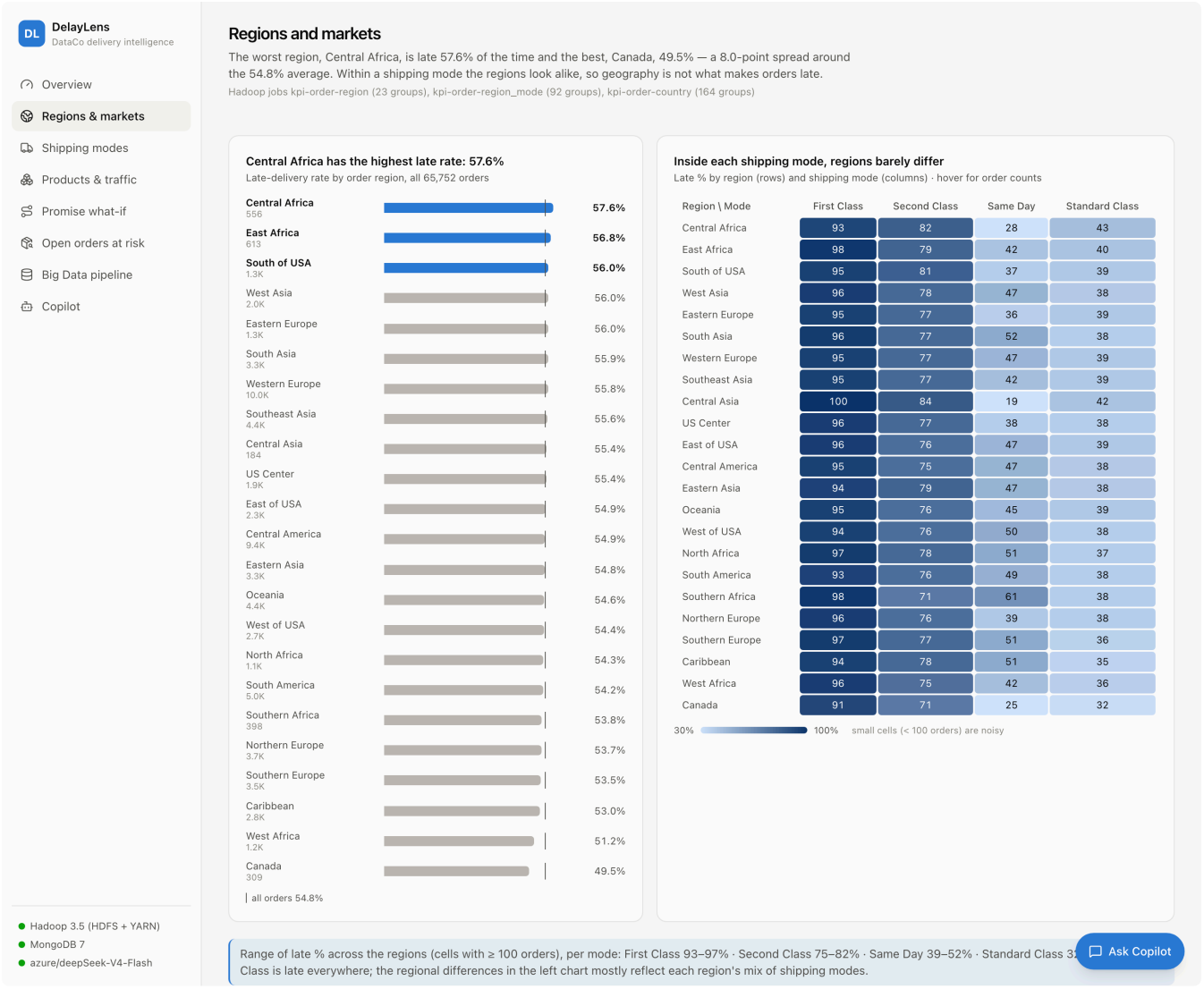



**Figure 8.4 Products and web traffic.** Late % by category and department, and page views by hour and weekday from the clickstream jobs.

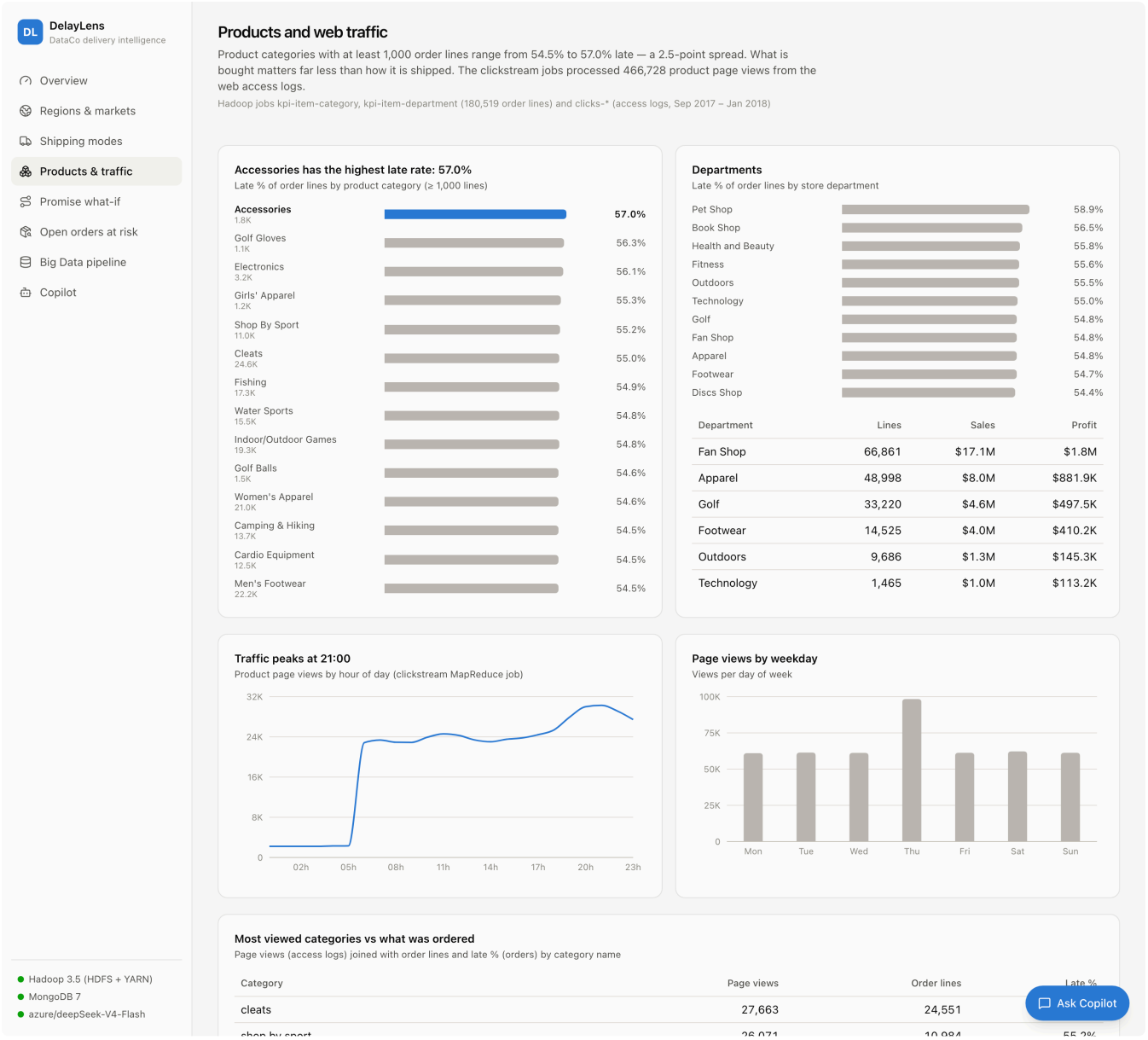



**Figure 8.5 Promise what-if.** Hadoop scenario results and the interactive policy simulator (here First Class 2 days, Second Class 4 days, Same Day cutoff 12:00: 54.8% to 30.3%).

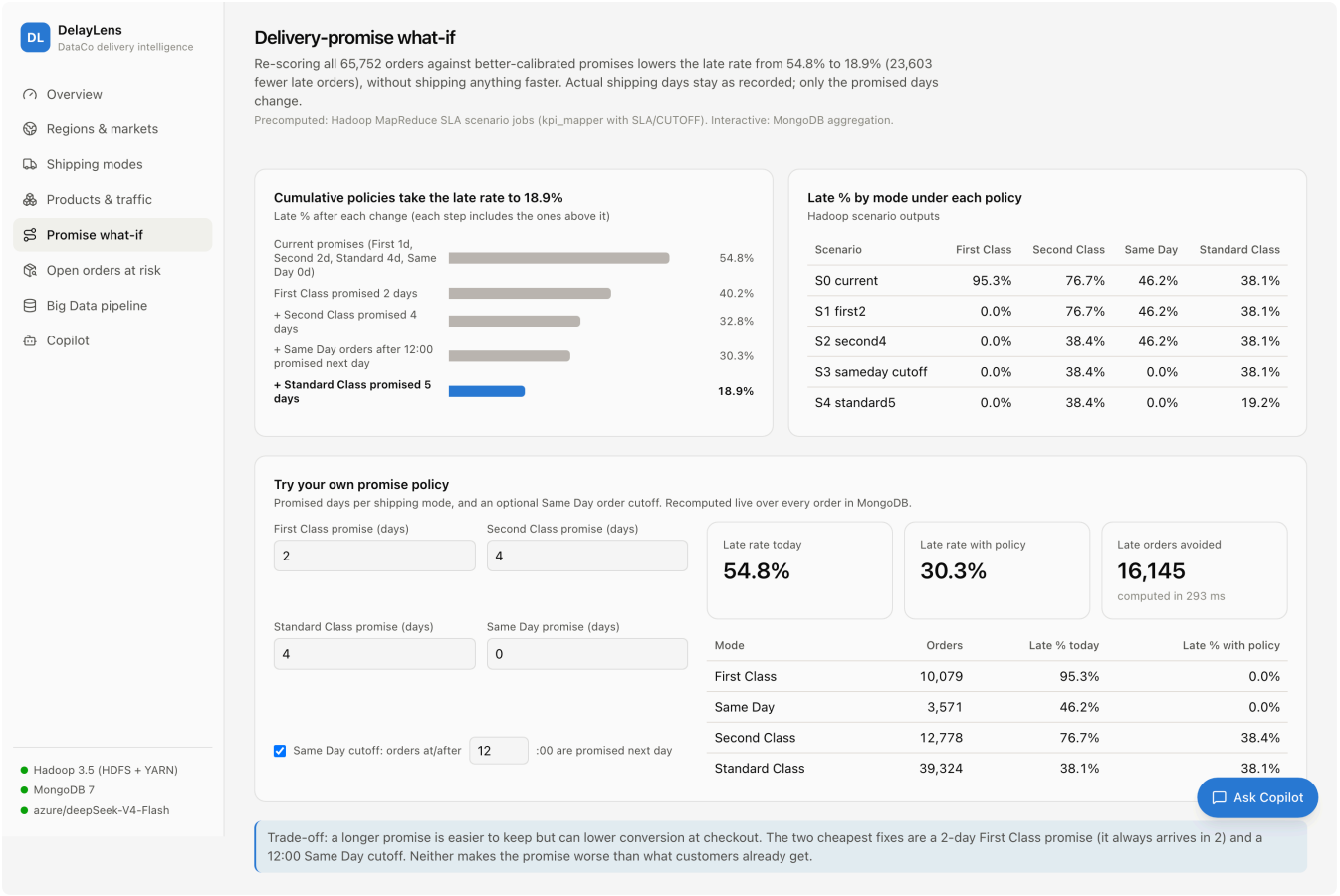



**Figure 8.6 Open orders at risk.** Filterable work list of open orders in segments that are usually late, with sales at stake.

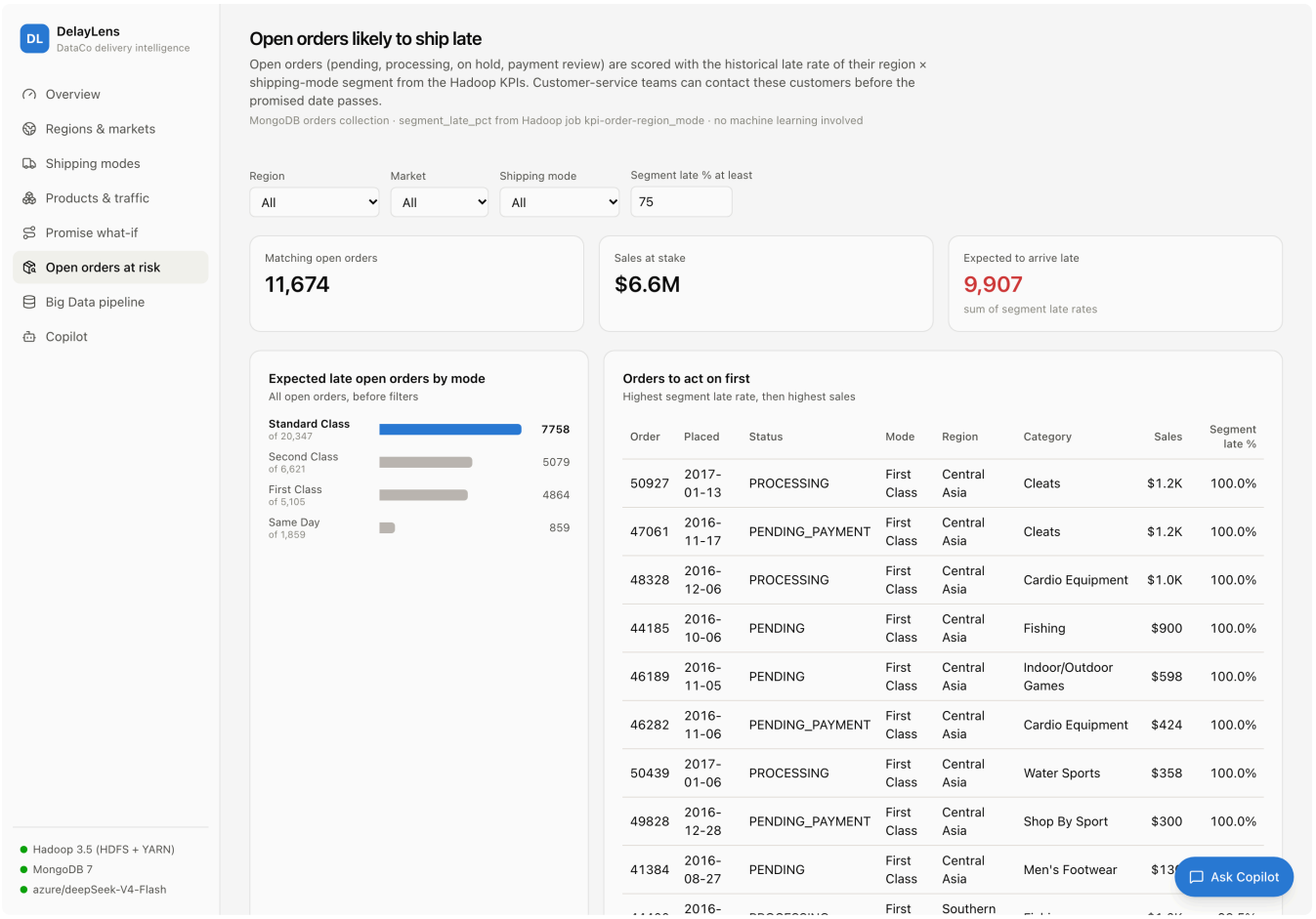



**Figure 8.7 Copilot.** The LangGraph agent answering a question: collapsible tool trace (6 tool calls), a table, an inline chart and recommendations, with the tool catalogue on the right.

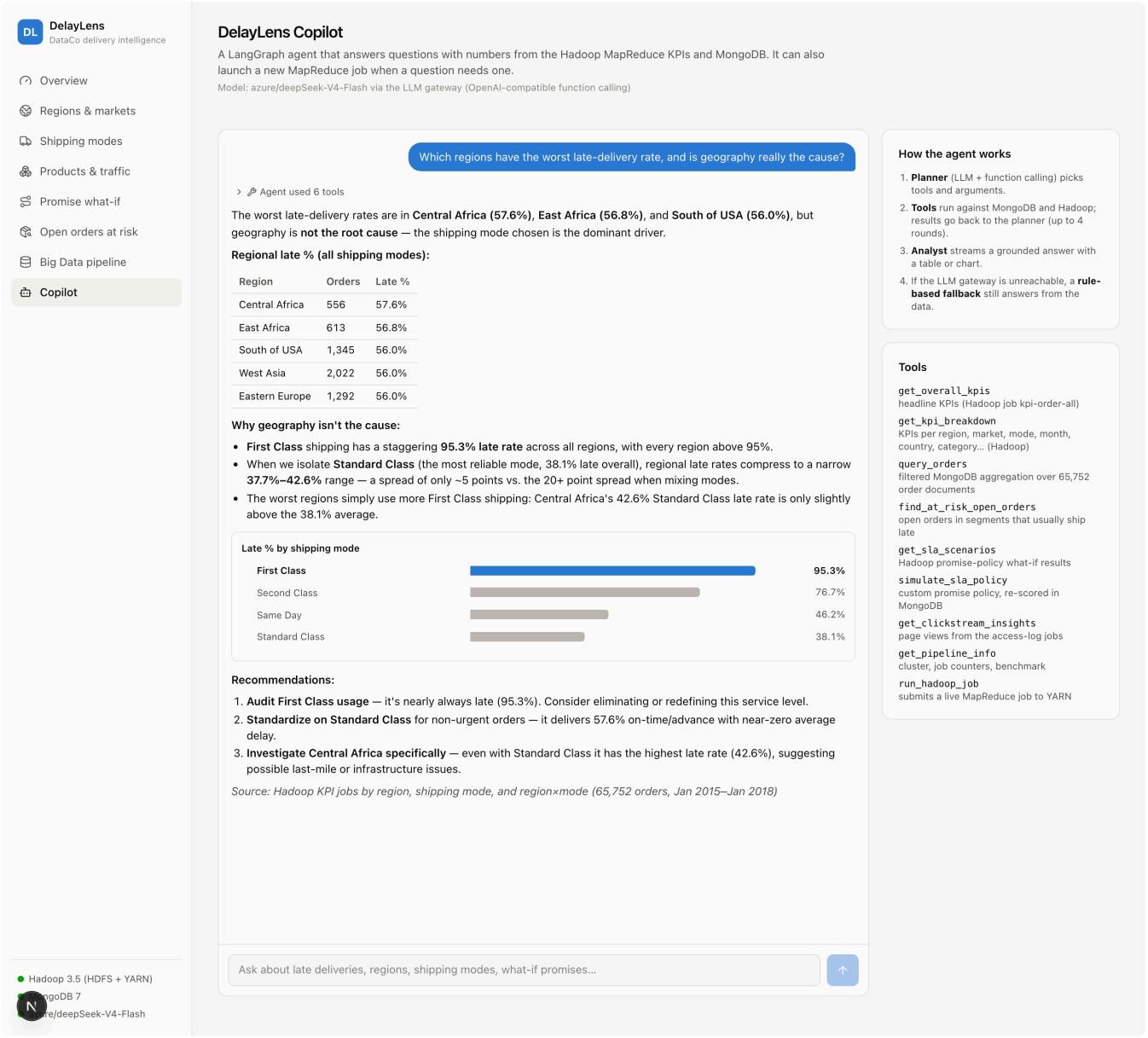
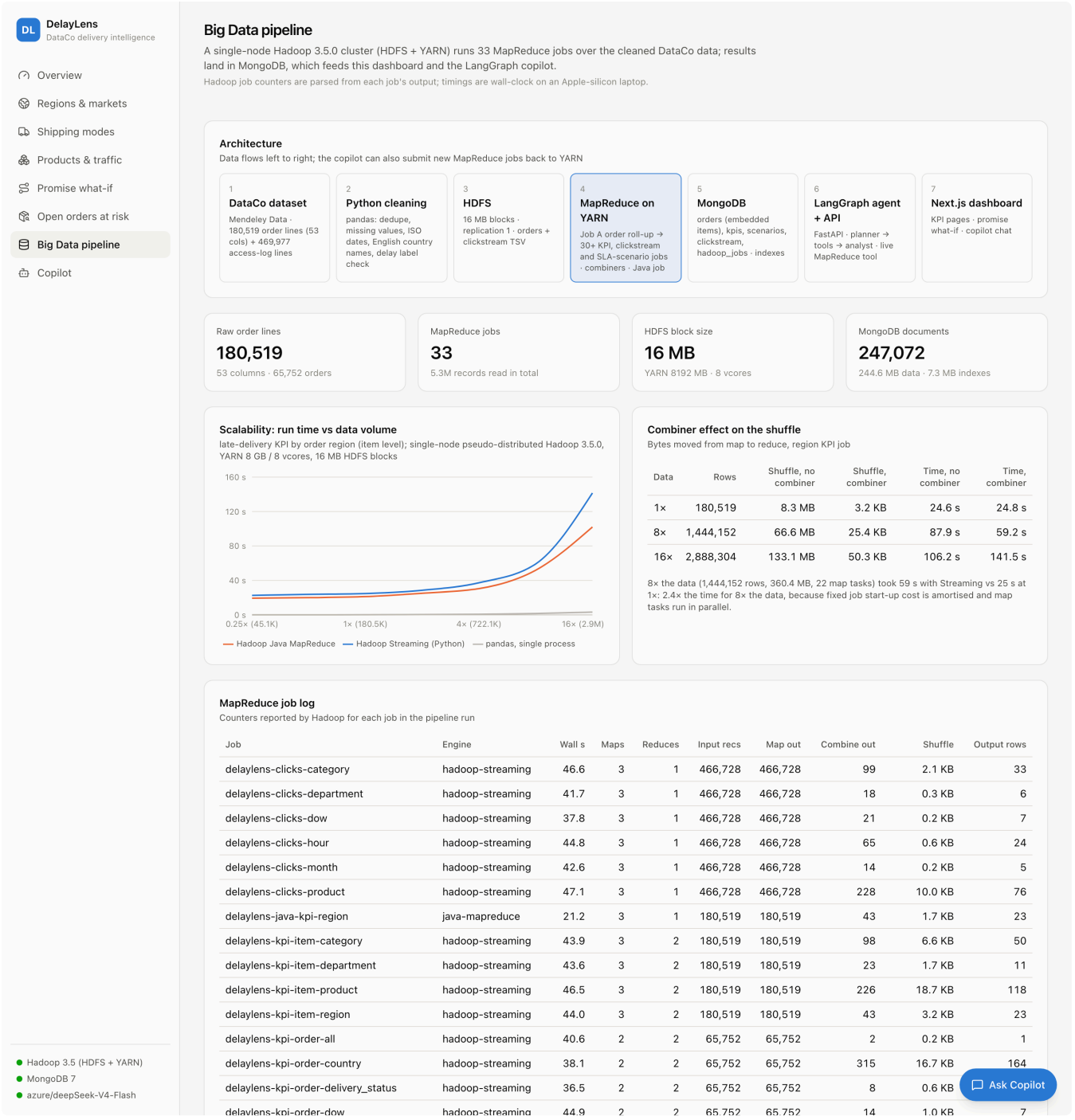



**Figure 8.8 Big Data pipeline.** Architecture, cluster facts, the scalability benchmark, the combiner effect and the counter log of every MapReduce job, read from the hadoop_jobs collection.




# 9. Conclusion

DataCo's 54.8% late-delivery rate comes from delivery promises that do not match how each shipping mode actually performs, not from geography or products. A Hadoop, MongoDB and agentic-AI pipeline found the cause, quantified the fix, and now answers follow-up questions in seconds.

## Key findings

1. 36,048 of 65,752 orders (54.8%) arrived late, and the rate held between 52.5% and 57.3% in every month from January 2015 to January 2018.

2. The shipping mode explains the problem. First Class is promised in 1 day but always takes 2 (95.3% late), Second Class is promised in 2 but takes 2 to 6 (76.7%), every Same Day order placed from 12:00 is late, and Standard Class does best (38.1%).
3. Region, market, product category, customer segment, weekday and month move the late rate by at most 8 points, and within a shipping mode regions differ even less. Payment type's 9-point gap is an artefact of where cancellations are recorded.
4. Three promise changes that never promise less than customers already receive would cut the late rate to 30.3% (16,145 fewer late orders); adding a 5-day Standard Class promise would bring it to 18.9%.
5. 11,674 open orders worth $6.58M sit in segments that are late at least 75% of the time.

**Recommendations for DataCo**

1. Promise 2 days for First Class now, or fix the carrier so it delivers in 1; today the premium service is late on every shipped order.
2. Quote Second Class at 4 days or negotiate a carrier service level. Actual times spread evenly over 2 to 6 days, which suggests Second Class gets no priority in the network.
3. Show a 12:00 Same Day cutoff at checkout and promise next-day delivery after it.
4. Treat a 5-day Standard Class promise as a commercial decision: it halves that mode's late rate (38.1% to 19.2%) but may lower conversion, so trial it in one market first.
5. Contact the 11,674 at-risk open orders before their date passes, highest sales first.
6. Re-run the MapReduce pipeline daily, track the KPIs on the dashboard, and use the copilot for ad-hoc questions instead of commissioning new reports.

**What the Big Data approach contributed.** Chained MapReduce (an ETL roll-up followed by 32 aggregation jobs) turned 650,000 raw records into 607 KPI rows. Combiners cut the shuffle about 2,600-fold. Run time grew 6.2x for 64x more data, with a fixed start-up cost of about 20 s per job. MongoDB's document model and indexes serve the results in milliseconds. LangGraph makes the copilot's steps explicit, and grounding rules plus an evaluation set kept its answers correct (8 of 8).

**Limitations**

- One single-node cluster: the shuffle never crosses a network, and each benchmark size was run once.
- The dataset is historical (2015 to 2018). Its even 2-to-6-day delivery spread and exact 12:00 cutoff suggest rule-generated data, so the findings describe DataCo's recorded process.
- The replicated benchmark data adds volume, not new information.
- The what-if analysis assumes actual delivery days stay the same when promises change, and it cannot measure the effect on conversion.

- The copilot's language model is reachable only on the company VPN; the rule-based fallback answers from the data without it.

**Future work.** Run the jobs on a multi-node cluster (for example Amazon EMR or Google Dataproc) and compare them with Spark. Expose the HDFS outputs as Hive tables for SQL users, and stream live orders in with Kafka. Add carrier-level data to explain the 2-to-6-day spread and conversion data to price the promise trade-off. Give the copilot memory and scheduled reports.

## 10. References


1. Constante, F., Silva, F., & Pereira, A. (2019). *DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS* (Version 5) [Data set]. Mendeley Data. CC BY 4.0. https://doi.org/10.17632/8gx2fvg2k6.5
2. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113. https://doi.org/10.1145/1327452.1327492
3. Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop Distributed File System. *IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*. https://doi.org/10.1109/MSST.2010.5496972
4. Vavilapalli, V. K., Murthy, A. C., Douglas, C., et al. (2013). Apache Hadoop YARN: Yet another resource negotiator. *Proceedings of the 4th ACM Symposium on Cloud Computing (SoCC '13)*. https://doi.org/10.1145/2523616.2523633
5. Apache Software Foundation. (2026). *Hadoop Streaming*, Apache Hadoop 3.5.0 documentation. https://hadoop.apache.org/docs/r3.5.0/hadoop-streaming/HadoopStreaming.html
6. MongoDB, Inc. (2026). *Aggregation Operations*, MongoDB Database Manual. https://www.mongodb.com/docs/manual/aggregation/
7. LangChain. (2026). *LangGraph overview*. https://docs.langchain.com/oss/python/langgraph/overview
8. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629
9. White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.
10. Apache Software Foundation. (2026). *Hadoop Java Versions*, Apache Hadoop wiki. https://cwiki.apache.org/confluence/display/HADOOP/Hadoop+Java+Versions
11. Apache Software Foundation. (2026). *Apache Hadoop releases* (3.5.0, 2 April 2026). https://hadoop.apache.org/releases.html